SEOS-2332 - Key Store workplan

Basic requirements

The Key Store module is intended to segregate the manipulation of sensitive cryptographic keys from the application which uses those keys. In this way, the top level application never has access to the content of the key but delegates its key management to the key store. Hence, the key store should store keys according to the application identifier and a specific name so that the application can require the usage of a given key it owns while other applications should not be authorized to have access to these.

Current features

We have two different things to separate here: the current status of the Key Store in **TRENTOS** and the current status of the **formally verified key store** module which is a standalone application.

Current TRENTOS Keystore

User level functions

A current user of the keystore is the **CryptoServer**. It initialises the Keystore library with:

```
initKeyStore(...)
```

The **Cryptoserver** itself is offering RPC calls to it's clients for loading and storing keys:

```
cryptoServer_rpc_loadKey(...)

cryptoServer_rpc_storeKey(...)
```

OS_CORE_API / OS_Network API

The TRENTOS SDK offers in OS_Keystore.h/.c an abstract user level programming interface for using a Keystore. The functions are:

- OS_Keystore_init()
- OS_Keystore_free()
- OS_Keystore_storeKey()
- OS_Keystore_loadKey()
- OS_Keystore_deleteKey()
- OS_Keystore_copyKey()
- OS_Keystore_moveKey()
- OS_Keystore_wipeKeystore()

Except OS_Keystore_init() and OS_Keystore_free() the functions are routed to a Keystore-Implementation, for which the TRENTOS SDK currently has one specific variant (KeystoreLib.h/.c)

OS_Keystore_init() takes as arguments handles to fileystem and crypto implementations, which are currently used in KeystoreLib.h/.c from the other functions this is hidden, and doesn't need to be used by a keystore implementation.

Implementation in KeystoreLib.c/.h

The single current implementation of the keystore in TRENTOS SDK provides the following functions:

```
KeystoreLib_storeKey(...)

KeystoreLib_loadKey(...)

KeystoreLib_deleteKey(...)

KeystoreLib_copyKey(...)

KeystoreLib_moveKey(...)

KeystoreLib_wipeKeystore(...)

KeystoreLib_init(...)

KeystoreLib_free(...)
```

Note that only the init, free, store load and delete key are currently used in the code., as only those are routed from the respective OS_Keystore_xxx() functions.

The current implementation stores and reads the keys to/from a file. For that it uses the filehandle given to `KeystoreLib_init(...)`

It uses the crypto handle to calculate and store a hash based on OS_CryptoDigest_ALG_SHA256 when a key is stored. On loading it recalculates and compares the hash.

There is also an AES on Non Volatile Memory module (AesNvm) which is never used and which does not differ from the current AES implementation. It also uses only an ECB mode which is not safe for key storage.

Current Formally Verified Keystore

The formally verified key store modules implements the following functions

```
void key_store_init (...)

key_store_wipe(...)

key_store_add(...)

key_store_get(...)

key_store_get_by_index(...)

key_store_delete(...)

key_store_init_with_read_only_keys(...)
```

The functions are self-explanatory with the exception of the last one. This was intended as an extra feature which prevents keys from being deleted by the apps and will not be used in the current iteration.

The formally verified keystore stores and loads keys only to/from RAM. It does not use any checksum or hashes. So it does not require at this time handles to crypto or filesystem like the KeystoreLib implementation.

The formally verified keystore has not been integrated with the OS_Keystore API. Especially implementations for KeystoreLib_init() and KeystoreLib_free() are missing.

Differences between keystore implementations features

- functions move key and copy key are present in the current variant but not in the formally verified one
    - move_key and cop_key move/copy between two instances of the keystore.
- Storage Backend
    - File for KeystoreLib
    - RAM for FV Keystore
- Hashes/Checkes Storage Backend
    - SHA256 hash for key data in for KeystoreLib
    - no checks for FV Keystore
- ...

- ...

Current implementation and possibilities of integration of formally verified library

The current implementation of the keystore library reads (stores) keys from (in) persistent memory using the File System library. The key store components included in the Crypto Server are based on the File System API as well which in turns creates a Storage component.
On the other hand, the formally verified key store library uses only RAM. This poses the problem of the persistent storage for the formally verified key store. The current status of the formally verified library is that the space it occupies in RAM can be adapted in the code so it can fit platform. That implies that every time the key store is used, the instance has to be loaded entirely using the FS library and then, once all modification / uses have been done, the key store has to be written back into persistent memory using the FS again.

It was discussed with the FV team that the next iteration will use the FS library functions supposing that that they are behaving like a standard memory allocation and read/write functions (this is possible by posing as an axiom their behavior).

Integration Alternatives

Current

direct implementation of implementation OS_Keystore.c/.h => KeystoreLib.c/h

Variant A - introduce FV Keystore as an alternative Keystore implementation

Pros

- FV Keystore can stay as it is (RAM based only), no watering down by introducing more code
- concept in other places as well, can be reused

Cons

- selection of keystore needs explanation
- have verified and unverified libraries

Open:

- concept for selecting the desired Keystore
  - by CMAKE project
  - by Define (OS_Keystore.c/.h is aware of implementations)
  - by parameter (OS_Keystore_init())
- optional parameters - handle to filesystem and crypto
  - keep as is
  - move to config or impl structure
- FV Keystore functionality
  - leave as is
  - add file support

Variant B - replace current implementation

Pros

- only one Keystore implementation in SDK
- easier implementation

Cons

- FV Keystore functionality < current Keystore

Open:

- FV Keystore functionality
  - leave as is
  - add file support
    - effect on formal verification unclear, as current concept is to take all persistent storage oriented functions as "known good" - that is a lot of code
- optional parameters - handle to filesystem and crypto
  - delete (in case not necessary any more)
  - keep as is

- move to config or impl structure

Other features

Missing interface functions

As mentioned earlier, the difference between the functionalities of the current and the new formally verified key store is in two functions: **copy_key** and **move_key**. If we want to keep those two functions with the formally verified library, we can easily do so as we can base them on existing functions. More specifically, **copy_key** is a **load_key** followed by a **store_key** with different app_id and **move_key** is a **load_key** followed by a **store_key** and a **delete_key**. One need to take into account, that those operations do involve two instances of the keystore.

File access

To add file access to the FV Keystore there are in principle two possibilities:

- add it directly to the functions, so data is not stored in RAM, but in a file or alternatively on some block device storage
  - This will add some code to the library that is not easily verifiable.
  - According to the FV Team such functions can be specified as axioms / "known good" in Isabelle and will "technically" not break the proof.
  - IMHO (TW) this will weaken the proof.
- keep the RAM based approach and do "backups to file"
  - this will add functions either in the FV Keystore library or in the wrapper code, that will do load from file or store to file operations in separate functions.
  - those functions are currently not foreseen in the OS_Keystore API => API change
  - Alternatively this could be hidden in the OS_Keystore_storeKey/OS_Keystore_deleteKey() by introducing another abstraction layer
    - OS_Keystore_storeKey() => FV_Keystore_store_wrapper() => `key_store_add()` + key_store_persist()
  - in the end this will end up in 2 synchronous storage places of the keystore, which might not be the best idea for a security oriented system.

Decision

In a meeting on 16.04.2021 the variants were presented and a decision for the the way forward was taken. The MOMs can be found here: 2021-04-16 - Alignment Formally Verified Keystore

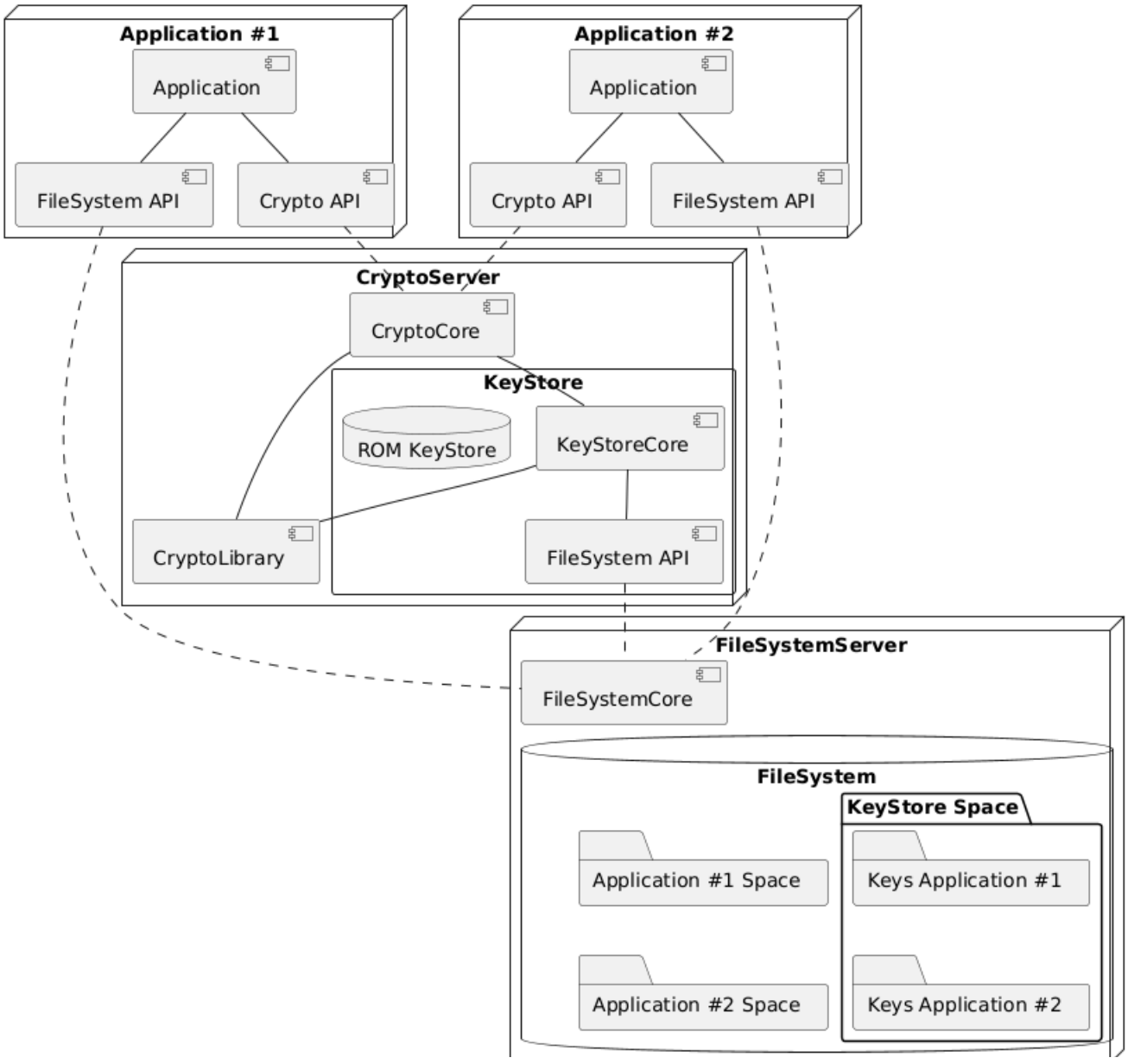
It was decided to perform the integration as follows:

- to include FV keystore as it is, without further feature extension, especially file storage
- integrate FV Keystore as a 2nd implementation of the Keystore API, in parallel to the already existing Keystore Implementation (KeyStoreLib.c/.h)

Further extension or Variants of the FV Keystore are seperate activities.

Other architecture topics with regards to TRENTOS

Currently, the key_store functionalities are embedded into crypto server following this diagram:


Application #1
Application
FileSystem API
Crypto API
Application #2
Application
Crypto API
FileSystem API
CryptoServer
CryptoCore
KeyStore
ROM KeyStore
KeyStoreCore
CryptoLibrary
FileSystem API
FileSystemServer
FileSystemCore
FileSystem
KeyStore Space
Application #1 Space
Keys Application #1
Application #2 Space
Keys Application #2

As a commentary, this forces any application which handles sensitive key information to pass by the crypto server. It should then be the case that the TLS server relies on a crypto server but there is no reference to a crypto_server in the TLS component. Instead, the TLS component calls directly the TLS functions of mbedTls wrapped into the OS_Tls library.

Proposition on extended features

The principal extension that we can propose to implement is the **key store encryption**. Indeed, segregating keys and preventing unauthorized applications to access key material means that the key are protected at run-time but are not protected against a memory dump or an access to physical memory b an attacker. The problem of key store encryption is that the **master key** used to encrypt all the other keys has to be stored somewhere on the device or to be derived from an user input. The solution consisting on key derivation from a user input password at boot time acts in a way like a Disk Encryption function. The solution of having the master key stored securely on the device requires a bit more effort on the hardware architecture but solutions exist like PUF-RAM or tamper-resistant write-only flash accessible only by the key store.

One can also have one dedicated master key per application. The problem remains essentially the same except that there will be one master key per application.

Proposition on architecture

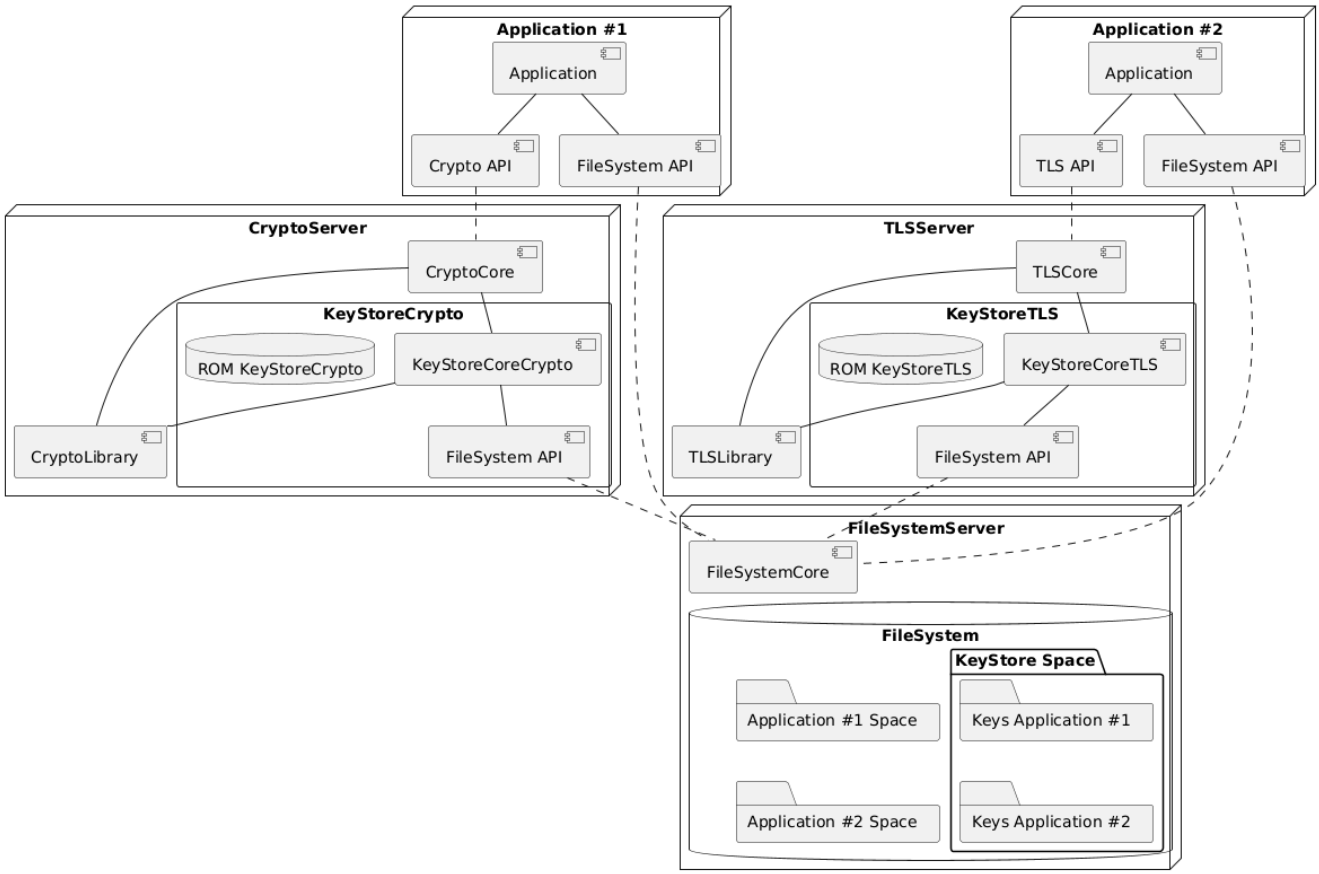
As mentioned earlier, the main problem with the current architecture is that the TLS server does not have access (or do not use) the key store capabilities. There are two ways to remedy this:

1. Integrate access to a keystore into the TLS server in the same fashion than in the current crypto server
2. Base the TLS server on a crypto server.
3. Decouple the key store server capabilities from the crypto server by creating a key store server which can be accessed by the TLS and crypto servers (as well as any future component which might need it, e.g. an encrypted file system).

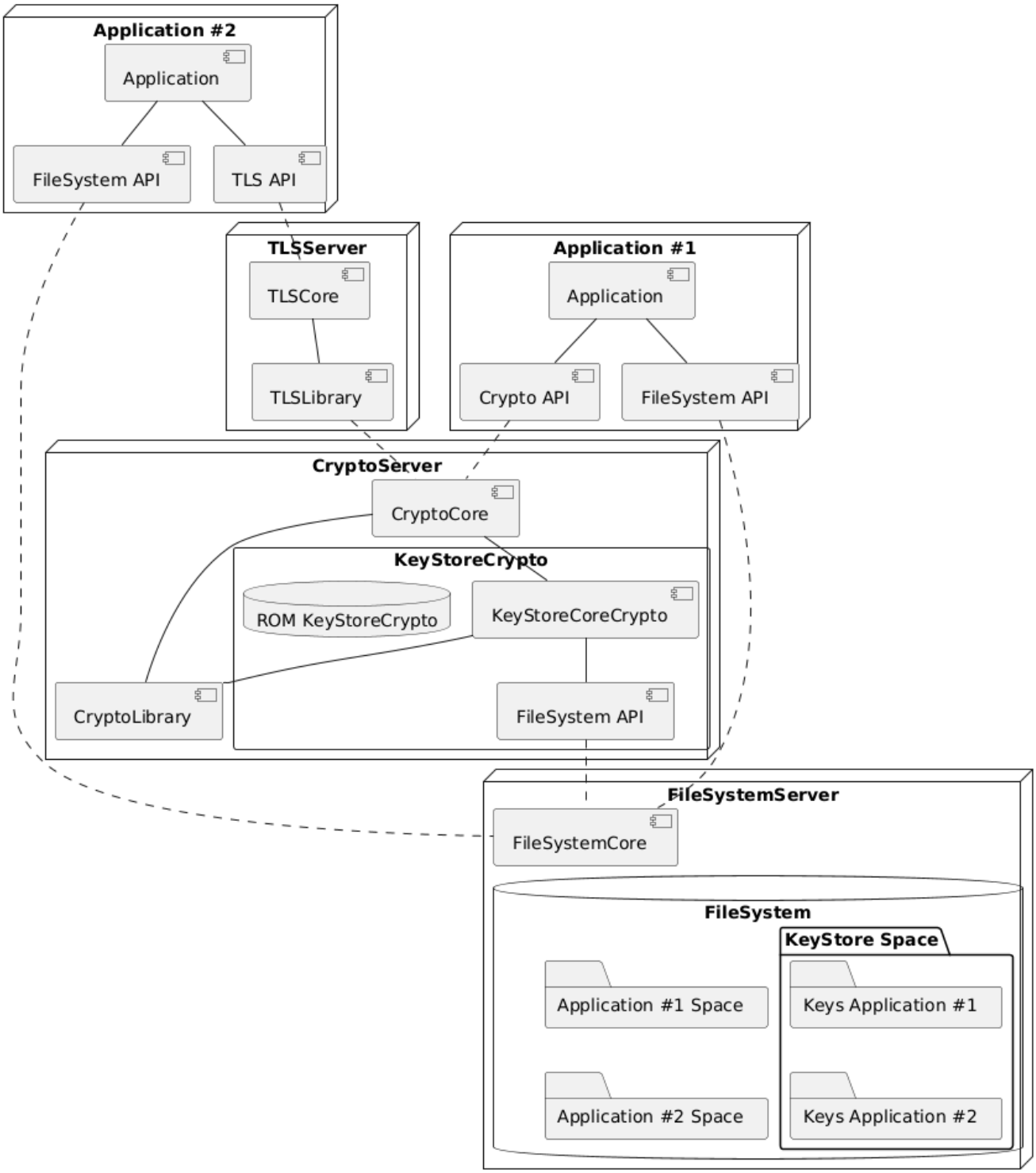
If the solution 1 or 2 is kept, we are to deal as many instances of key stores as crypto and TLS servers. If solution 3 is retained, there is the possibility of having a unique key store defined for the whole system.

We summarize the 4 possible combinations in the architecture diagrams below. We show a corresponding workflow for key access corresponding to each solution. Note that for all those architectures, enabling encrypted storage and storing a master key is a different problem as the key store should be able to access the master key on a memory where only it has access.

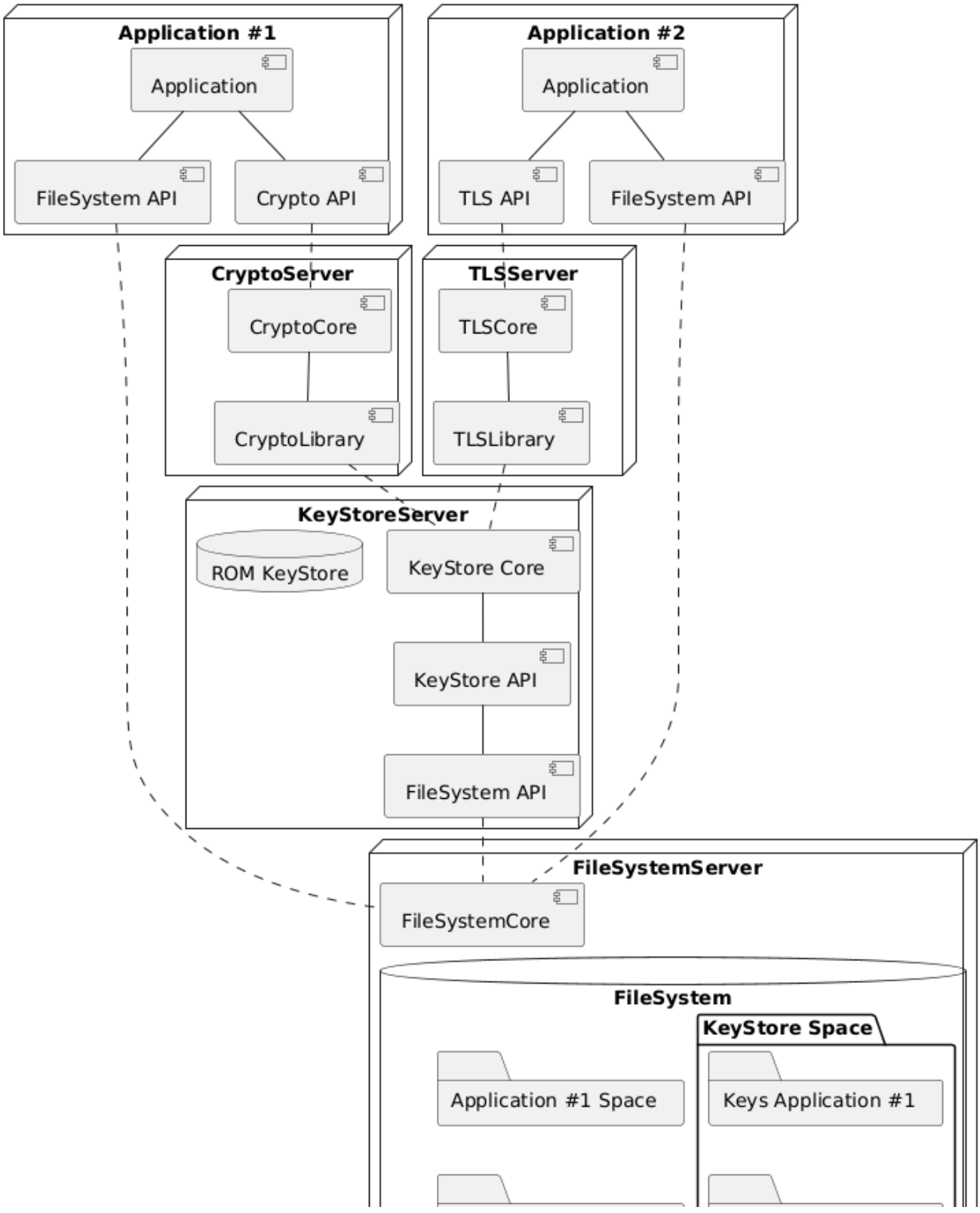
Solution 1



Solution 2

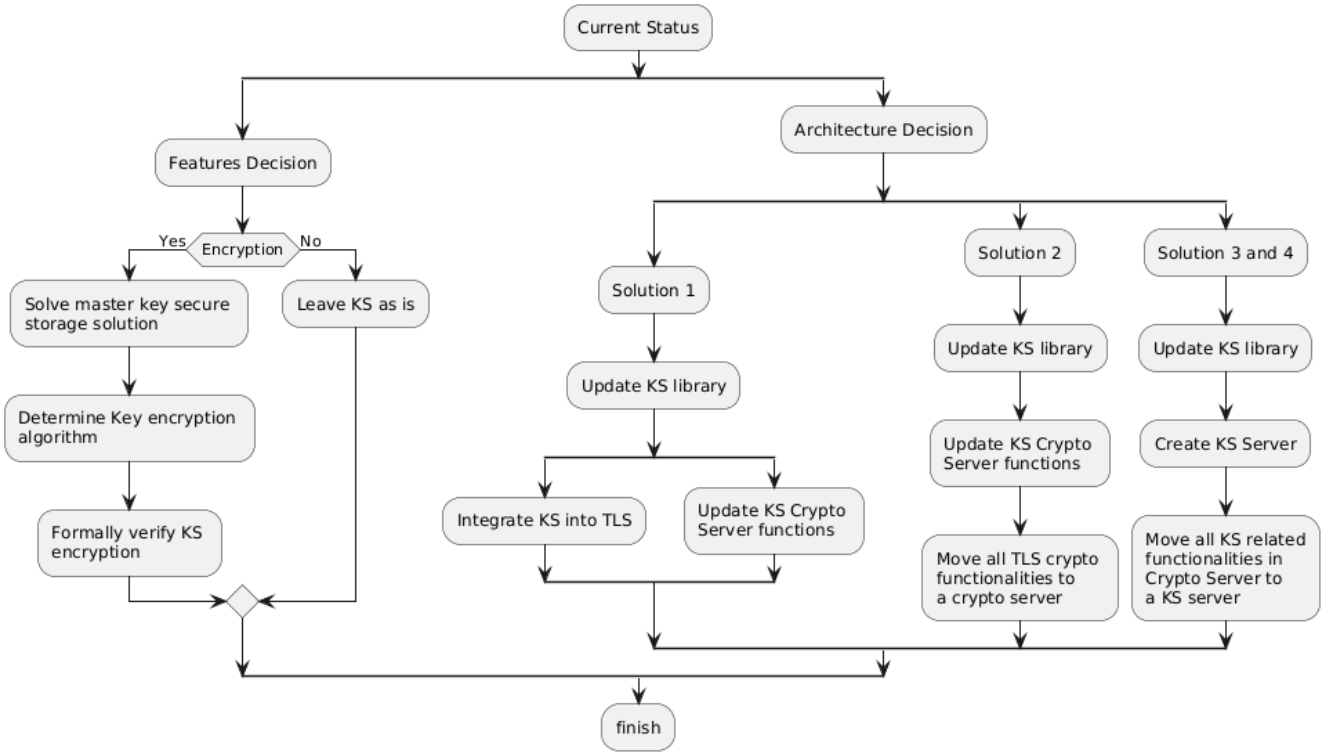


Solution 3 (and 4)


Application #1
Application
FileSystem API
Crypto API
Application #2
Application
TLS API
FileSystem API
CryptoServer
CryptoCore
CryptoLibrary
TLSServer
TLSCore
TLSLibrary
KeyStoreServer
ROM KeyStore
KeyStore Core
KeyStore API
FileSystem API
FileSystemServer
FileSystemCore
FileSystem
KeyStore Space
Application #1 Space
Keys Application #1

Note that solution 2 raises the independent question of rebasing the TLS server on a crypto server

Work-plan



The work-plan is summarized in the following epic in Jira:

Key	Summary	T	Created	Updated	Due	Assignee	Reporter	P	Status	Resolution

Authenticate to retrieve your issues

No issues found